

QRUIT — Deployment Guide

Getting the project live on one public URL. No CI/CD here — just the steps to stand up a single working server.

What you're standing up

Piece	What it is	Runs as
Postgres	the database	system service
Redis	job queue for background work (screening, interviews, alerts)	system service
Backend API	FastAPI app, uvicorn	systemd service, port 8787
Worker	processes the background jobs	systemd service (arq app.worker.WorkerSettings)
Frontend	React app	built once to static files, served by nginx
nginx	reverse proxy + HTTPS	system service

■ You need two hostnames, not one

This is the one non-obvious part — read this before touching DNS.

- **Public hostname** (e.g. `qrquit.yourdomain.com`) — the only thing the internet talks to. Handles the Google/Microsoft OAuth callback and the WhatsApp webhook.
- **Admin hostname** (e.g. `admin.yourdomain.com`, or keep it on a VPN / IP-restricted) — serves the frontend and everything under `/api/*` (connecting channels, managing roles).

The backend deliberately **rejects** `/api/*` requests if the Host header matches the public hostname (`app/observability.py::PublicHostnameGuardMiddleware`) — that's intentional, not a bug. If the frontend is put on the same domain as `QRUIT_PUBLIC_BASE_URL`, every API call will 403. Two server names (they can point at the same machine) is the fix.

1. Server prep (Ubuntu, adjust for your distro)

```
sudo apt update && sudo apt install -y python3.11 python3.11-venv postgresql \
redis-server nginx certbot python3-certbot-nginx nodejs npm
sudo -u postgres createuser qrquit -P          # set a real password
sudo -u postgres createdb qrquit -O qrquit
```

2. Backend

```
git clone <repo-url> /opt/qrquit && cd /opt/qrquit/backend
python3.11 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
```

Edit .env:

- QRUIT_PUBLIC_BASE_URL → https://qruii.yourdomain.com (the public hostname)
- CORS_ALLOWED_ORIGINS → https://admin.yourdomain.com (the admin hostname)
- DATABASE_URL → postgresql+psycopg://qruii:<password>@localhost:5432/qruii
- REDIS_URL → redis://localhost:6379/0
- QRUIT_APPROVAL_SECRET → generate with: `python -c "import secrets; print(secrets.token_urlsafe(48))"`
- leave every provider field (GOOGLE_*, MS_*, WHATSAPP_*) blank for now — connect those later from the frontend's Channels page, using its Setup Guide tab
- flip QRUIT_EMAIL MOCK / QRUIT MOCK API / QRUIT INTERVIEW MOCK / WHATSAPP MOCK to false once real credentials are in, one at a time

Run the database migration:

```
alembic upgrade head
```

Two systemd services — /etc/systemd/system/qruii-api.service:

```
[Unit]
Description=QRUIT API
After=network.target postgresql.service redis-server.service

[Service]
WorkingDirectory=/opt/qruii/backend
ExecStart=/opt/qruii/backend/.venv/bin/uvicorn app.main:app --host 127.0.0.1 --port 8787
Restart=always
User=qruii

[Install]
WantedBy=multi-user.target
```

/etc/systemd/system/qruii-worker.service — same block but:

```
ExecStart=/opt/qruii/backend/.venv/bin/arq app.worker.WorkerSettings
sudo systemctl enable --now qruii-api qruii-worker
```

3. Frontend

```
cd /opt/qruii/frontend
echo "VITE_API_BASE_URL=https://admin.yourdomain.com/api-proxy" > .env
npm install
npm run build          # outputs static files to dist/
```

The VITE_API_BASE_URL value above assumes the nginx setup below, which proxies /api-proxy on the admin domain through to the backend — that's what keeps the frontend's calls landing on the *admin* Host header, not the public one.

4. nginx — two server blocks

```
# Public – OAuth callback + WhatsApp webhook only
server {
    server_name qruit.yourdomain.com;
    location / {
        proxy_pass http://127.0.0.1:8787;
        proxy_set_header Host $host;
    }
}

# Admin – frontend + API
server {
    server_name admin.yourdomain.com;

    root /opt/qruit/frontend/dist;
    location / { try_files $uri /index.html; }

    location /api-proxy/ {
        rewrite ^/api-proxy/(.*)$ /$1 break;
        proxy_pass http://127.0.0.1:8787;
        proxy_set_header Host admin.yourdomain.com;
    }
}
```

Then HTTPS for both:

```
sudo certbot --nginx -d qruit.yourdomain.com -d admin.yourdomain.com
sudo systemctl reload nginx
```

5. Point DNS

Create A records for both qruit.yourdomain.com and admin.yourdomain.com pointing at the server's IP.

6. Verify

```
curl https://qruit.yourdomain.com/healthz      # backend is up
```

Open <https://admin.yourdomain.com> in a browser — you should see the QRUIT Channels page. Use its **Setup Guide** tab to connect Gmail/Outlook/etc.